AFL 模糊测试实验

一、实验环境:

Ubuntu 18.04-64bit

二、实验准备

1. 下载并安装 AFL。

下载地址：https://github.com/google/AFL

（1） 按照默认提供的安装方法进行安装，安装完成后 afl-gcc 和 afl-fuzz 均可正常使用。

Code Blame Raw

```
========================
Installation instructions
========================

  This document provides basic installation instructions and discusses known
  issues for a variety of platforms. See README for the general instruction
  manual.

1) Linux on x86
---------------

This platform is expected to work well. Compile the program with:

$ make

You can start using the fuzzer without installation, but it is also possible to
install it with:

# make install

There are no special dependencies to speak of; you will need GNU make and a
working compiler (gcc or clang). Some of the optional scripts bundled with the
program may depend on bash, gdb, and similar basic tools.

If you are using clang, please review llvm_mode/README.llvm; the LLVM
integration mode can offer substantial performance gains compared to the
traditional approach.

You may have to change several settings to get optimal results (most notably,
disable crash reporting utilities and switch to a different CPU governor), but
afl-fuzz will guide you through that if necessary.
```

cd AFL-master

make # 编译 AFL

sudo make install # 在系统中安装 AFL

```
ray@ubuntu:~/AFL-master$ sudo make install
[sudo] password for ray:
[*] Checking for the ability to compile x86 code...
[+] Everything seems to be working, ready to compile.
[*] Testing the CC wrapper and instrumentation output...
unset AFL_USE_ASAN AFL_USE_MSAN; AFL_QUIET=1 AFL_INST_RATIO=100 AFL_PATH=. ./afl-gcc -O3 -funroll-loop
s -Wall -D_FORTIFY_SOURCE=2 -g -Wno-pointer-sign -DAFL_PATH=\"/usr/local/lib/afl\" -DDOC_PATH=\"/usr/l
ocal/share/doc/afl\" -DBIN_PATH=\"/usr/local/bin\" test-instr.c -o test-instr -ldl
./afl-showmap -m none -q -o .test-instr0 ./test-instr < /dev/null
echo 1 | ./afl-showmap -m none -q -o .test-instr1 ./test-instr
[+] All right, the instrumentation seems to be working!
[+] All done! Be sure to review README - it's pretty short and useful.


mkdir -p -m 755 ${DESTDIR}/usr/local/bin ${DESTDIR}/usr/local/lib/afl ${DESTDIR}/usr/local/share/doc/a
fl ${DESTDIR}/usr/local/share/afl
rm -f ${DESTDIR}/usr/local/bin/afl-plot.sh
install -m 755 afl-gcc afl-fuzz afl-showmap afl-tmin afl-gotcpu afl-analyze afl-plot afl-cmin afl-what
sup ${DESTDIR}/usr/local/bin
rm -f ${DESTDIR}/usr/local/bin/afl-as
if [ -f afl-qemu-trace ]; then install -m 755 afl-qemu-trace ${DESTDIR}/usr/local/bin; fi
if [ -f afl-clang-fast -a -f afl-llvm-pass.so -a -f afl-llvm-rt.o ]; then set -e; install -m 755 afl-c
lang-fast ${DESTDIR}/usr/local/bin; ln -sf afl-clang-fast ${DESTDIR}/usr/local/bin/afl-clang-fast++; i
nstall -m 755 afl-llvm-pass.so afl-llvm-rt.o ${DESTDIR}/usr/local/lib/afl; fi
if [ -f afl-llvm-rt-32.o ]; then set -e; install -m 755 afl-llvm-rt-32.o ${DESTDIR}/usr/local/lib/afl;
 fi
if [ -f afl-llvm-rt-64.o ]; then set -e; install -m 755 afl-llvm-rt-64.o ${DESTDIR}/usr/local/lib/afl;
 fi
set -e; for i in afl-g++ afl-clang afl-clang++; do ln -sf afl-gcc ${DESTDIR}/usr/local/bin/$i; done
install -m 755 afl-as ${DESTDIR}/usr/local/lib/afl
ln -sf afl-as ${DESTDIR}/usr/local/lib/afl/as
install -m 644 README.md docs/ChangeLog docs/*.txt ${DESTDIR}/usr/local/share/doc/afl
cp -r testcases/ ${DESTDIR}/usr/local/share/afl
cp -r dictionaries/ ${DESTDIR}/usr/local/share/afl
ray@ubuntu:~/AFL-master$
```

验证 afl-gcc 和 afl-fuzz 均可正常使用：

```
ray@ubuntu: ~/AFL-master
ray@ubuntu:~/AFL-master$ afl-gcc
afl-cc 2.57b by <lcamtuf@google.com>

This is a helper application for afl-fuzz. It serves as a drop-in replacement
for gcc or clang, letting you recompile third-party code with the required
runtime instrumentation. A common use pattern would be one of the following:

  CC=/usr/local/bin/afl-gcc ./configure
  CXX=/usr/local/bin/afl-g++ ./configure

You can specify custom next-stage toolchain via AFL_CC, AFL_CXX, and AFL_AS.
Setting AFL_HARDEN enables hardening optimizations in the compiled code.

ray@ubuntu:~/AFL-master$ afl-fuzz
afl-fuzz 2.57b by <lcamtuf@google.com>

afl-fuzz [ options ] -- /path/to/fuzzed_app [ ... ]

Required parameters:

  -i dir        - input directory with test cases
  -o dir        - output directory for fuzzer findings

Execution control settings:

  -f file       - location read by the fuzzed program (stdin)
  -t msec       - timeout for each run (auto-scaled, 50-1000 ms)
  -m megs       - memory limit for child process (50 MB)
  -Q            - use binary-only instrumentation (QEMU mode)

Fuzzing behavior settings:

  -d            - quick & dirty mode (skips deterministic steps)
  -n            - fuzz without instrumentation (dumb mode)
  -x dir        - optional fuzzer dictionary (see README)

Other stuff:

  -T text       - text banner to show on the screen
  -M / -S id    - distributed mode (see parallel_fuzzing.txt)
  -C            - crash exploration mode (the peruvian rabbit thing)
  -V            - show version number and exit

  -b cpu_id     - bind the fuzzing process to the specified CPU core

For additional tips, please consult /usr/local/share/doc/afl/README.

ray@ubuntu:~/AFL-master$
```

（2）进入 qemu_mode 目录，使用 build_qemu_support.sh 脚本构建 AFL-Qemu。（可能需安装 libtool, libtool-bin 等依赖）。如 AFL-Qemu 安装存在问题，还可预先安装 qemu。

安装 libtool, libtool-bin 等：apt 安装对应包

Qemu 安装原链接失效：在 https://download.qemu.org/ 直接下载 qemu 2.10.0.tar.xz

```
ray@ubuntu: ~/AFL-master/q
ray@ubuntu:~/AFL-master/qemu_mode$ ./build_qemu_support.sh
==================================================
AFL binary-only instrumentation QEMU build script
==================================================

[*] Performing basic sanity checks...
[+] All checks passed!
[+] Cryptographic signature on qemu-2.10.0.tar.xz checks out.
[*] Uncompressing archive (this will take a while)...
[+] Unpacking successful.
[*] Configuring QEMU for ...
[*] Applying patches...
patching file linux-user/elfload.c
patching file accel/tcg/cpu-exec.c
patching file linux-user/syscall.c
patching file configure
patching file util/memfd.c
[+] Patching done.
Install prefix    /usr/local
BIOS directory    /usr/local/share/qemu
binary directory  /usr/local/bin
library directory /usr/local/lib
module directory  /usr/local/lib/qemu
libexec directory /usr/local/libexec
include directory /usr/local/include
config directory  /usr/local/etc
local state directory   /usr/local/var
Manual directory  /usr/local/share/man
ELF interp prefix /usr/gnemul/qemu-%M
Source path       /home/ray/AFL-master/qemu_mode/qemu-2.10.0
C compiler        cc
Host C compiler   cc
C++ compiler      c++
Objective-C compiler cc
ARFLAGS           rv
CFLAGS            -O2 -U_FORTIFY_SOURCE -D_FORTIFY_SOURCE=2 -g -O3 -ggdb
QEMU_CFLAGS       -I$(SRC_PATH)/pixman/pixman -I$(BUILD_DIR)/pixman/pixman -I$(SRC_PATH)/dtc/libfdt -p
thread -I/usr/include/glib-2.0 -I/usr/lib/x86_64-linux-gnu/glib-2.0/include -fPIE -DPIE -m64 -mcx16 -D
_GNU_SOURCE -D_FILE_OFFSET_BITS=64 -D_LARGEFILE_SOURCE -Wstrict-prototypes -Wredundant-decls -Wall -Wu
ndef -Wwrite-strings -Wmissing-prototypes -fno-strict-aliasing -fno-common -fwrapv  -Wexpansion-to-def
ined -Wendif-labels -Wno-shift-negative-value -Wno-missing-include-dirs -Wempty-body -Wnested-externs
-Wformat-security -Wformat-y2k -Winit-self -Wignored-qualifiers -Wold-style-declaration -Wold-style-de
finition -Wtype-limits -fstack-protector-strong -I/usr/include/libusb-1.0
LDFLAGS           -Wl,--warn-common -Wl,-z,relro -Wl,-z,now -pie -m64 -g
make              make
install           install
python            python2.7 -B
smbd              /usr/sbin/smbd
module support    no
host CPU          x86_64
host big endian   no
```

出现以上说明安装成功

```
ray@ubuntu: ~/AFL-master/q
In file included from /home/ray/AFL-master/qemu_mode/qemu-2.10.0/accel/tcg/cpu-exec.c:39:0:
/home/ray/AFL-master/qemu_mode/qemu-2.10.0/../patches/afl-qemu-cpu-inl.h:107:19: warning: redundant re
declaration of 'tb_htable_lookup' [-Wredundant-decls]
 TranslationBlock *tb_htable_lookup(CPUState*, target_ulong, target_ulong, uint32_t);
                   ^~~~~~~~~~~~~~~~~
In file included from /home/ray/AFL-master/qemu_mode/qemu-2.10.0/accel/tcg/cpu-exec.c:23:0:
/home/ray/AFL-master/qemu_mode/qemu-2.10.0/include/exec/exec-all.h:403:19: note: previous declaration
of 'tb_htable_lookup' was here
 TranslationBlock *tb_htable_lookup(CPUState *cpu, target_ulong pc,
                   ^~~~~~~~~~~~~~~~~
  CC      x86_64-linux-user/accel/tcg/cpu-exec-common.o
  CC      x86_64-linux-user/accel/tcg/translate-all.o
  CC      x86_64-linux-user/linux-user/main.o
  CC      x86_64-linux-user/linux-user/syscall.o
  CC      x86_64-linux-user/linux-user/strace.o
  CC      x86_64-linux-user/linux-user/mmap.o
  CC      x86_64-linux-user/linux-user/signal.o
  CC      x86_64-linux-user/linux-user/elfload.o
  CC      x86_64-linux-user/linux-user/linuxload.o
  CC      x86_64-linux-user/linux-user/uaccess.o
  CC      x86_64-linux-user/linux-user/uname.o
  CCAS    x86_64-linux-user/linux-user/safe-syscall.o
  CC      x86_64-linux-user/target/i386/helper.o
  CC      x86_64-linux-user/target/i386/cpu.o
  CC      x86_64-linux-user/target/i386/gdbstub.o
  CC      x86_64-linux-user/target/i386/xsave_helper.o
  CC      x86_64-linux-user/target/i386/translate.o
  CC      x86_64-linux-user/target/i386/bpt_helper.o
  CC      x86_64-linux-user/target/i386/cc_helper.o
  CC      x86_64-linux-user/target/i386/excp_helper.o
  CC      x86_64-linux-user/target/i386/fpu_helper.o
  CC      x86_64-linux-user/target/i386/int_helper.o
  CC      x86_64-linux-user/target/i386/mem_helper.o
  CC      x86_64-linux-user/target/i386/misc_helper.o
  CC      x86_64-linux-user/target/i386/mpx_helper.o
  CC      x86_64-linux-user/target/i386/seg_helper.o
  CC      x86_64-linux-user/target/i386/smm_helper.o
  CC      x86_64-linux-user/target/i386/svm_helper.o
  CC      x86_64-linux-user/target/i386/kvm-stub.o
  GEN     trace/generated-helpers.c
  CC      x86_64-linux-user/trace/generated-helpers.o
  CC      x86_64-linux-user/trace/control-target.o
  LINK    x86_64-linux-user/qemu-x86_64
[+] Build process successful!
[*] Copying binary...
-rwxrwxr-x 1 ray ray 10968040 May 11 09:49 ../afl-qemu-trace
[+] Successfully created '../afl-qemu-trace'.
[*] Testing the build...
[+] Instrumentation tests passed.
[+] All set, you can now use the -Q mode in afl-fuzz!
ray@ubuntu:~/AFL-master/qemu_mode$
```

2. 下载目标程序集（coreutils-9.1.tar.gz）。

下载地址：https://ftp.gnu.org/gnu/coreutils/

三、实验步骤

1. 基于编译器的目标程序插桩

（1）使用 afl-gcc，生成 coreutils 的每个二进制程序，具体步骤为：

a）查看 coreutils 的 configure 选项，指定 cc=afl-gcc。

cd coreutils-9.1

./configure CC=afl-gcc

```
ray@ubuntu:~/coreutils-9.1$ ./configure CC=afl-gcc
checking for a BSD-compatible install... /usr/bin/install -c
checking whether build environment is sane... yes
checking for a race-free mkdir -p... /bin/mkdir -p
checking for gawk... no
checking for mawk... mawk
checking whether make sets $(MAKE)... yes
checking whether make supports nested variables... yes
checking whether make supports nested variables... (cached) yes
checking for gcc... afl-gcc
checking whether the C compiler works... yes
checking for C compiler default output file name... a.out
checking for suffix of executables...
checking whether we are cross compiling... no
checking for suffix of object files... o
checking whether the compiler supports GNU C... yes
checking whether afl-gcc accepts -g... yes
checking for afl-gcc option to enable C11 features... none needed
checking whether afl-gcc understands -c and -o together... yes
checking whether the compiler is clang... no
checking for compiler option needed when checking for declarations... none
checking whether make supports the include directive... yes (GNU style)
checking dependency style of afl-gcc... gcc3
checking how to run the C preprocessor... afl-gcc -E
checking for grep that handles long lines and -e... /bin/grep
checking for egrep... /bin/grep -E
checking whether afl-gcc needs -traditional... no
checking for ranlib... ranlib
checking whether ln -s works... yes
checking for stdio.h... yes
checking for stdlib.h... yes
checking for string.h... yes
checking for inttypes.h... yes
checking for stdint.h... yes
checking for strings.h... yes
checking for sys/stat.h... yes
checking for sys/types.h... yes
checking for unistd.h... yes
checking for wchar.h... yes
checking for minix/config.h... no
checking for arpa/inet.h... yes
checking for features.h... yes
checking for sys/socket.h... yes
```

b）生成 coreutils 二进制程序集。make 出来的每一个插桩后二进制均在 coreutils-9.1/src 目录下。

Make

```
ray@ubuntu: ~/coreutils-9.1
afl-as 2.57b by <lcamtuf@google.com>
[+] Instrumented 10 locations (64-bit, non-hardened mode, ratio 100%).
 CC       lib/libcoreutils_a-closeout.o
afl-cc 2.57b by <lcamtuf@google.com>
afl-as 2.57b by <lcamtuf@google.com>
[+] Instrumented 12 locations (64-bit, non-hardened mode, ratio 100%).
 CC       lib/libcoreutils_a-copy-file-range.o
afl-cc 2.57b by <lcamtuf@google.com>
afl-as 2.57b by <lcamtuf@google.com>
[+] Instrumented 15 locations (64-bit, non-hardened mode, ratio 100%).
 CC       lib/libcoreutils_a-count-leading-zeros.o
afl-cc 2.57b by <lcamtuf@google.com>
afl-as 2.57b by <lcamtuf@google.com>
[+] Instrumented 9 locations (64-bit, non-hardened mode, ratio 100%).
 CC       lib/libcoreutils_a-af_alg.o
afl-cc 2.57b by <lcamtuf@google.com>
afl-as 2.57b by <lcamtuf@google.com>
[!] WARNING: No instrumentation targets found.
 CC       lib/libcoreutils_a-md5-stream.o
afl-cc 2.57b by <lcamtuf@google.com>
afl-as 2.57b by <lcamtuf@google.com>
[+] Instrumented 14 locations (64-bit, non-hardened mode, ratio 100%).
 CC       lib/libcoreutils_a-md5.o
afl-cc 2.57b by <lcamtuf@google.com>
afl-as 2.57b by <lcamtuf@google.com>
[+] Instrumented 141 locations (64-bit, non-hardened mode, ratio 100%).
 CC       lib/libcoreutils_a-sha1-stream.o
afl-cc 2.57b by <lcamtuf@google.com>
afl-as 2.57b by <lcamtuf@google.com>
[+] Instrumented 14 locations (64-bit, non-hardened mode, ratio 100%).
 CC       lib/libcoreutils_a-sha1.o
afl-cc 2.57b by <lcamtuf@google.com>
afl-as 2.57b by <lcamtuf@google.com>
[+] Instrumented 141 locations (64-bit, non-hardened mode, ratio 100%).
 CC       lib/libcoreutils_a-sha256-stream.o
afl-cc 2.57b by <lcamtuf@google.com>
afl-as 2.57b by <lcamtuf@google.com>
[+] Instrumented 30 locations (64-bit, non-hardened mode, ratio 100%).
 CC       lib/libcoreutils_a-sha256.o
afl-cc 2.57b by <lcamtuf@google.com>
afl-as 2.57b by <lcamtuf@google.com>
[+] Instrumented 188 locations (64-bit, non-hardened mode, ratio 100%).
 CC       lib/libcoreutils_a-sha512-stream.o
afl-cc 2.57b by <lcamtuf@google.com>
afl-as 2.57b by <lcamtuf@google.com>
[+] Instrumented 17 locations (64-bit, non-hardened mode, ratio 100%).
 CC       lib/libcoreutils_a-sha512.o
afl-cc 2.57b by <lcamtuf@google.com>
afl-as 2.57b by <lcamtuf@google.com>
[+] Instrumented 103 locations (64-bit, non-hardened mode, ratio 100%).
```

（2）为 coreutils 的特定程序确定输入种子

coreutils 包含很多二进制程序，选择 3 个程序:cat.c,cp.c,du.c 进行 fuzzing。构造输入种子列表，存放于 coreutils-9.1/src/input 目录下，构造方法可以选择：

a）oreutils 文档中这 3 个程序的命令行选项，用这些命令行选项构建

```python
import os
import random

# sort 命令可用的选项列表
sort_options = [
    "-r",               # reverse
    "-n",               # numeric sort
    "-u",               # unique
    "-k1,1",            # sort on key 1
    "--numeric-sort",
    "--reverse",
    "-M",               # sort by month
    "-V",               # version sort
    "-b",               # ignore leading blanks
    "-f",               # fold case
]

# 配置输出参数
NUM_FILES = 50
MAX_OPTIONS_PER_CMD = 4
OUTPUT_DIR = "input_sort"
os.makedirs(OUTPUT_DIR, exist_ok=True)

for i in range(NUM_FILES):
    input_line = " "
    # 随机选出 1 到 MAX_OPTIONS_PER_CMD 个参数
    selected_options = random.sample(sort_options, random.randint(1, MAX_OPTIONS_PER_CMD))
    selected_data = f" random_bytes/data_{random.randint(0,9)}.bin"
    # 把它们组成一行参数字符串（AFL 会传给程序的 argv 或 stdin）
    for option in selected_options:
        input_line+=f" {str(option)}"
    # 保存到输入文件
    input_line+=selected_data
    print(input_line)
    with open(os.path.join(OUTPUT_DIR, f"id_{i:04d}"), "w") as f:
        f.write(input_line + "\n")

print(f"✅ 成功生成 {NUM_FILES} 个 sort 命令参数输入文件在 `{OUTPUT_DIR}/` 中。")
```

```
ray@ubuntu:~/coreutils-9.1/src$ cat input_sort/id_0000
  -u --reverse -n random_bytes/data_5.bin
ray@ubuntu:~/coreutils-9.1/src$ cat input_sort/id_0001
  -k1,1 random_bytes/data_1.bin
ray@ubuntu:~/coreutils-9.1/src$ cat input_sort/id_0002
  -u random_bytes/data_8.bin
ray@ubuntu:~/coreutils-9.1/src$ cat input_sort/id_0003
  -n -r -M -V random_bytes/data_1.bin
ray@ubuntu:~/coreutils-9.1/src$
```

b）直接用随机构造的任意字符串作为输入种子。

```
ray@ubuntu: ~/coreutils-9.1/s
GNU nano 2.9.3                    generate_random_bytes.py

import os
import random

# 配置参数
NUM_FILES = 10              # 要生成的文件数量
MIN_SIZE = 128              # 最小文件大小（字节）
MAX_SIZE = 4096             # 最大文件大小（字节）
OUTPUT_DIR = "byte_inputs"  # 输出目录

# 创建输出目录
os.makedirs(OUTPUT_DIR, exist_ok=True)

for i in range(NUM_FILES):
    size = random.randint(MIN_SIZE, MAX_SIZE)
    random_bytes = os.urandom(size)  # 生成随机字节流

    filename = os.path.join(OUTPUT_DIR, f"data_{i}.bin")
    with open(filename, "wb") as f:
        f.write(random_bytes)

print(f"✅ 生成了 {NUM_FILES} 个随机字节流文件在 `{OUTPUT_DIR}/` 文件夹中。")
```

（3）在 coreutils-9.1/src 目录下，使用“afl-fuzz -i input -o output ./[程序名] @@”进行 fuzzing，一段时间后终止 fuzzing 并在 coreutils-9.1/src/output 目录下查看测试结果。

报错，按照提示:

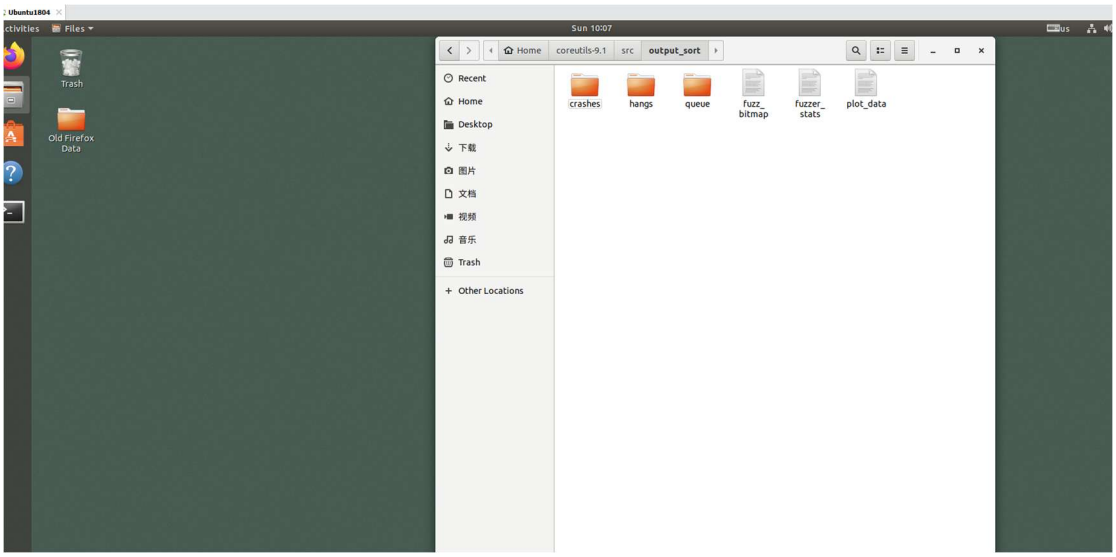
su

echo core >proc/sys/kernel/core_pattern

重新 afl-fuzz

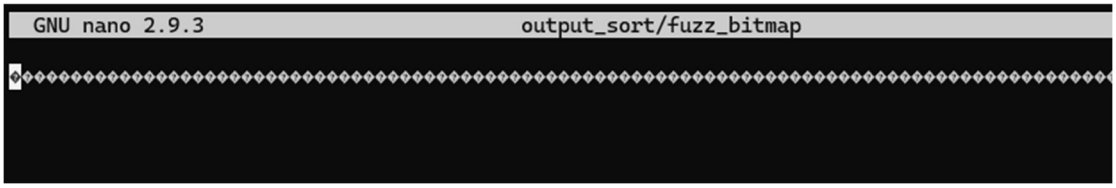
```
ray@ubuntu: ~/coreutils-9.1/s

                    american fuzzy lop 2.57b (sort)

┌─ process timing ──────────────────────────────────────┬─ overall results ─────┐
│        run time : 0 days, 0 hrs, 0 min, 17 sec        │  cycles done : 0      │
│   last new path : 0 days, 0 hrs, 0 min, 2 sec         │  total paths : 73     │
│ last uniq crash : none seen yet                       │ uniq crashes : 0      │
│  last uniq hang : none seen yet                       │   uniq hangs : 0      │
├─ cycle progress ────────────────────┬─ map coverage ─┴───────────────────────┤
│  now processing : 13 (17.81%)       │    map density : 0.52% / 0.74%         │
│ paths timed out : 0 (0.00%)         │ count coverage : 2.22 bits/tuple       │
├─ stage progress ────────────────────┼─ findings in depth ────────────────────┤
│  now trying : havoc                 │ favored paths : 13 (17.81%)            │
│ stage execs : 39/1024 (3.81%)       │  new edges on : 24 (32.88%)            │
│ total execs : 34.8k                 │ total crashes : 0 (0 unique)           │
│  exec speed : 1981/sec              │  total tmouts : 0 (0 unique)           │
├─ fuzzing strategy yields ───────────┴───────────────┬─ path geometry ────────┤
│   bit flips : 1/80, 0/78, 0/74                      │    levels : 2          │
│  byte flips : 0/10, 0/8, 0/4                        │   pending : 72         │
│ arithmetics : 2/556, 0/119, 0/0                     │  pend fav : 12         │
│  known ints : 0/42, 0/220, 0/176                    │ own finds : 63         │
│  dictionary : 0/0, 0/0, 0/0                         │  imported : n/a        │
│       havoc : 60/32.8k, 0/0                         │ stability : 100.00%    │
│        trim : 70.59%/8, 0.00%                       ├────────────────────────┘
└─────────────────────────────────────────────────────┘          [cpu000: 61%]
```

cp 和 uniq 同上



Output 里生成 fuzz_bitmap 和 fuzz_stats 和 plot_data

fuzz_bitmap 是 AFL 表示路径覆盖率信息的位图；fuzz_stats 记录 fuzzing 的实时统计信息；plot_data 是时间序列状态日志，可用于画图。



sort 的 fuzz_stats 如下

```
  GNU nano 2.9.3                    output_sort/fuzzer_stats

start_time        : 1746928775
last_update       : 1746929344
fuzzer_pid        : 115467
cycles_done       : 1
execs_done        : 876549
execs_per_sec     : 1182.53
paths_total       : 185
paths_favored     : 19
paths_found       : 175
paths_imported    : 0
max_depth         : 8
cur_path          : 180
pending_favs      : 0
pending_total     : 150
variable_paths    : 0
stability         : 100.00%
bitmap_cvg        : 0.80%
unique_crashes    : 0
unique_hangs      : 0
last_path         : 1746929315
last_crash        : 0
last_hang         : 0
execs_since_crash : 876549
exec_timeout      : 20
afl_banner        : sort
afl_version       : 2.57b
target_mode       : default
command_line      : afl-fuzz -i input_sort/ -o output_sort/ ./sort @@
slowest_exec_ms   : 11
peak_rss_mb       : 3
```

2. 基于 AFL-Qemu 的目标程序动态插桩

（1）重新生成 coreutils 的每个二进制程序（不使用 afl-gcc，而是用默认 gcc 进行 configure/make）。

./configure

make

（2）使用 afl-fuzz 的-Q 选项，对 3 个 coreutils 程序进行 fuzzing。构造输入种子的方法同上。

echo "ghjhgkjgkgt" > input/seed2

afl-fuzz -Q -i input -o output ./cat @@

```
root@blc-virtual-machine:/home/blc/桌面/coreutils-9.1/src# afl-fuzz -Q -i input -o output-- /bin/cat @@
afl-fuzz 2.57b by <lcamtuf@google.com>
[+] You have 2 CPU cores and 3 runnable tasks (utilization: 150%).
[*] Checking CPU core loadout...
[+] Found a free CPU core, binding to #0.
[*] Checking core_pattern...
[*] Setting up output directories...
[*] Scanning 'input'...
[+] No auto-generated dictionary tokens to reuse.
[*] Creating hard links for all input files...
[*] Validating target binary...

[-] Oops, unable to find the 'afl-qemu-trace' binary. The binary must be built
    separately by following the instructions in qemu_mode/README.qemu. If you
    already have the binary installed, you may need to specify AFL_PATH in the
    environment.

    Of course, even without QEMU, afl-fuzz can still work with binaries that are
    instrumented at compile time with afl-gcc. It is also possible to use it as a
    traditional "dumb" fuzzer by specifying '-n' in the command line.

[-] PROGRAM ABORT : Failed to locate 'afl-qemu-trace'.
         Location : get_qemu_argv(), afl-fuzz.c:7741
```

报错不能定位到 afl-qemu-trace，解决方法是把 afl-qemu-trace 加入到环境变量

AFL_PATH=/home/blc/桌面/AFL-master afl-fuzz -Q -i input -o output -- /bin/cat @@

```
oot@blc-virtual-machine:/home/blc/桌面/coreutils-9.1/src# AFL_PATH=/home/blc/桌面/AFL-master afl-fuzz -Q -i input -o output -- /bin/cat @@
fl-fuzz 2.57b by <lcamtuf@google.com>
+] You have 2 CPU cores and 5 runnable tasks (utilization: 250%).
!] WARNING: System under apparent load, performance may be spotty.
*] Checking CPU core loadout...
+] Found a free CPU core, binding to #0.
*] Checking core_pattern...
*] Setting up output directories...
+] Output directory exists but deemed OK to reuse.
*] Deleting old session data...
+] Output dir cleanup successful.
*] Scanning 'input'...
+] No auto-generated dictionary tokens to reuse.
*] Creating hard links for all input files...
*] Validating target binary...
*] Attempting dry run with 'id:000000,orig:seed1'...
*] Spinning up the fork server...
+] All right - fork server is up.
   len = 15, map size = 125, exec speed = 6241 us
*] Attempting dry run with 'id:000001,orig:seed2'...
   len = 12, map size = 125, exec speed = 1564 us
!] WARNING: No new instrumentation output, test case may be useless.
+] All test cases processed.

!] WARNING: Some test cases look useless. Consider using a smaller set.
+] Here are some useful stats:
```

```
                    american fuzzy lop 2.57b (cat)

┌─ process timing ─────────────────────────────────────┬─ overall results ─────┐
│        run time : 0 days, 0 hrs, 0 min, 33 sec       │  cycles done : 11     │
│   last new path : none yet (odd, check syntax!)      │  total paths : 2      │
│ last uniq crash : none seen yet                      │ uniq crashes : 0      │
│  last uniq hang : none seen yet                      │   uniq hangs : 0      │
├─ cycle progress ────────────────────┬─ map coverage ─┴───────────────────────┤
│  now processing : 1 (50.00%)        │    map density : 0.19% / 0.19%         │
│ paths timed out : 0 (0.00%)         │ count coverage : 1.00 bits/tuple       │
├─ stage progress ────────────────────┼─ findings in depth ────────────────────┤
│  now trying : splice 6              │ favored paths : 1 (50.00%)             │
│ stage execs : 13/48 (27.08%)        │  new edges on : 1 (50.00%)             │
│ total execs : 20.8k                 │ total crashes : 0 (0 unique)           │
│  exec speed : 660.8/sec             │  total tmouts : 0 (0 unique)           │
├─ fuzzing strategy yields ───────────┴───────────────┬─ path geometry ────────┤
│   bit flips : 0/64, 0/62, 0/58                      │    levels : 1          │
│  byte flips : 0/8, 0/6, 0/2                         │   pending : 0          │
│ arithmetics : 0/448, 0/0, 0/0                       │  pend fav : 0          │
│  known ints : 0/45, 0/168, 0/88                     │ own finds : 0          │
│  dictionary : 0/0, 0/0, 0/0                         │  imported : n/a        │
│       havoc : 0/8448, 0/11.4k                       │ stability : 100.00%    │
│        trim : 70.37%/5, 0.00%                       ├────────────────────────┘
└─────────────────────────────────────────────────────┘          [cpu000: 57%]
^C

+++ Testing aborted by user +++
[+] We're done here. Have a nice day!

root@blc-virtual-machine:/home/blc/桌面/coreutils-9.1/src#
```

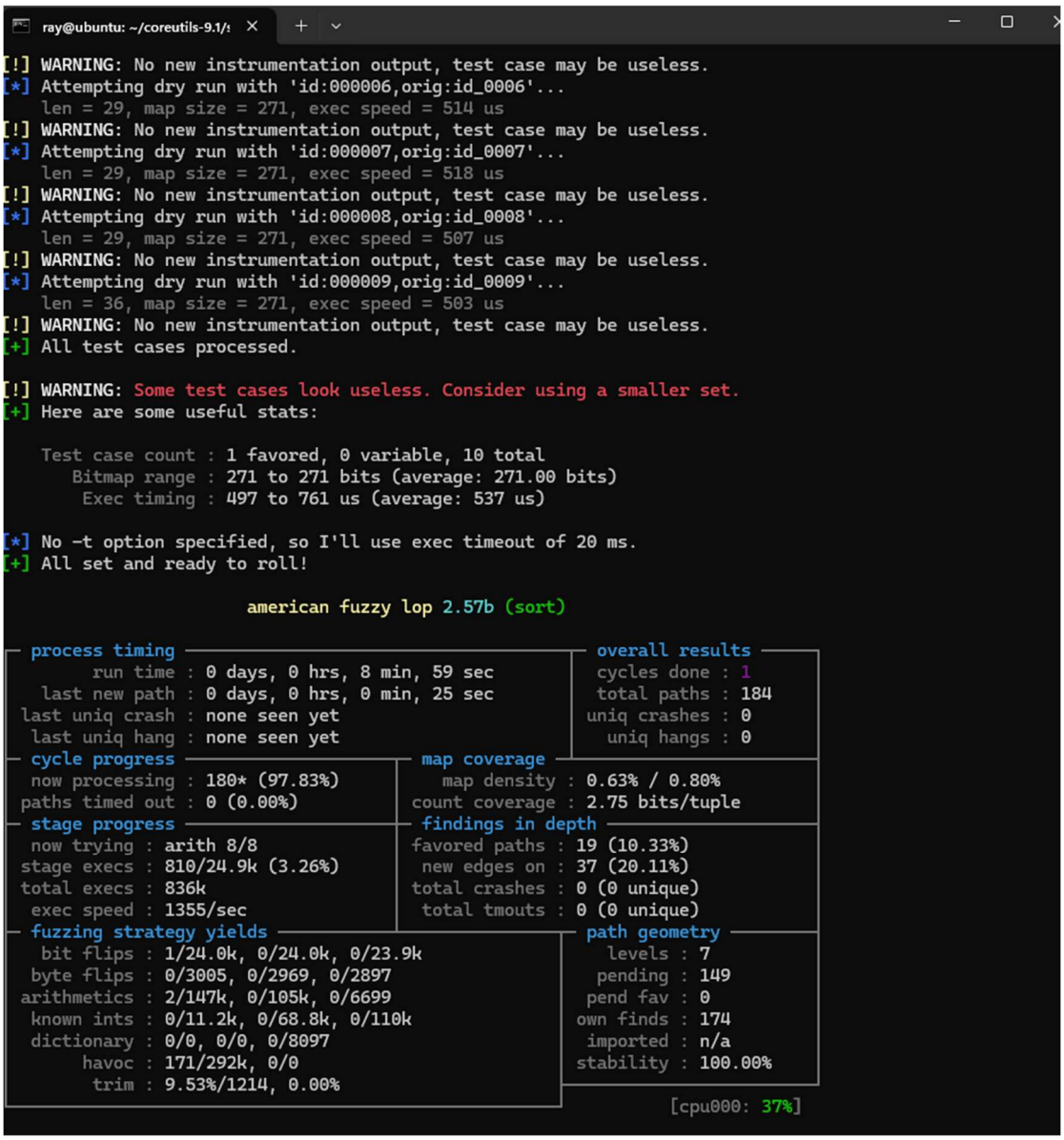
```
                    american fuzzy lop 2.57b (cp)

┌─ process timing ─────────────────────────────────────┬─ overall results ─────┐
│        run time : 0 days, 0 hrs, 0 min, 7 sec        │  cycles done : 1      │
│   last new path : none yet (odd, check syntax!)      │  total paths : 2      │
│ last uniq crash : none seen yet                      │ uniq crashes : 0      │
│  last uniq hang : none seen yet                      │   uniq hangs : 0      │
├─ cycle progress ────────────────────┬─ map coverage ─┴───────────────────────┤
│  now processing : 0* (0.00%)        │    map density : 0.37% / 0.37%         │
│ paths timed out : 0 (0.00%)         │ count coverage : 1.00 bits/tuple       │
├─ stage progress ────────────────────┼─ findings in depth ────────────────────┤
│  now trying : havoc                 │ favored paths : 1 (50.00%)             │
│ stage execs : 210/768 (27.34%)      │  new edges on : 1 (50.00%)             │
│ total execs : 2717                  │ total crashes : 0 (0 unique)           │
│  exec speed : 358.3/sec             │  total tmouts : 0 (0 unique)           │
├─ fuzzing strategy yields ───────────┴───────────────┬─ path geometry ────────┤
│   bit flips : 0/64, 0/62, 0/58                      │    levels : 1          │
│  byte flips : 0/8, 0/6, 0/2                         │   pending : 1          │
│ arithmetics : 0/448, 0/0, 0/0                       │  pend fav : 0          │
│  known ints : 0/45, 0/168, 0/88                     │ own finds : 0          │
│  dictionary : 0/0, 0/0, 0/0                         │  imported : n/a        │
│       havoc : 0/1536, 0/0                           │ stability : 100.00%    │
│        trim : 70.37%/5, 0.00%                       ├────────────────────────┘
^C                                                            [cpu000: 65%]

+++ Testing aborted by user +++
[+] We're done here. Have a nice day!

root@blc-virtual-machine:/home/blc/桌面/coreutils-9.1/src#
```

将 cp 改成 du 显示找不到文件

```
root@blc-virtual-machine:/home/blc/桌面/coreutils-9.1/src# AFL_PATH=/home/blc/桌面/AFL-master afl-fuzz -Q -i input -o output -- /bin/du @@
afl-fuzz 2.57b by <lcamtuf@google.com>
[+] You have 2 CPU cores and 1 runnable tasks (utilization: 50%).
[+] Try parallel jobs - see /usr/local/share/doc/afl/parallel_fuzzing.txt.
[*] Checking CPU core loadout...
[+] Found a free CPU core, binding to #0.
[*] Checking core_pattern...
[*] Setting up output directories...
[+] Output directory exists but deemed OK to reuse.
[*] Deleting old session data...
[+] Output dir cleanup successful.
[*] Scanning 'input'...
[+] No auto-generated dictionary tokens to reuse.
[*] Creating hard links for all input files...
[*] Validating target binary...

[-] PROGRAM ABORT : Program '/bin/du' not found or not executable
         Location : check_binary(), afl-fuzz.c:6890
```

用 which du 发现路径变为/usr/bin/du 在指令中也修改

```
root@blc-virtual-machine:/home/blc/桌面/coreutils-9.1/src# AFL_PATH=/home/blc/桌面/AFL-master afl-fuzz -Q -i input -o output -- /usr/bin/du @@
afl-fuzz 2.57b by <lcamtuf@google.com>
[+] You have 2 CPU cores and 3 runnable tasks (utilization: 150%).
[*] Checking CPU core loadout...
[+] Found a free CPU core, binding to #0.
[*] Checking core_pattern...
[*] Setting up output directories...
[+] Output directory exists but deemed OK to reuse.
[*] Deleting old session data...
[+] Output dir cleanup successful.
```



四、问题思考

（2）第“三、1（2）”步骤的两种输入种子构造方法，对于 fuzzing 结果的影响（是否有影响？如果有，对最终获得的路径数量、路径数增长的速度等方面的具体影响）。

有明显影响。使用有结构、有语义的命令行选项作为输入（方法 a），稳定线性增长，可能缓慢减少，但仍持续发现新路径，使最终路径数量更多和路径增长速度更快。

使用随机字符串（方法 b）基本上只能覆盖命令解析入口和报错分支，前期快速增长几个路径，然后趋于停滞，增长缓慢。

（3）第“三、1”基于编译插桩的 fuzzing 与第“三、2”的基于 Qemu 动态插桩的 fuzzing，从基本原理和实验效果两方面的差异。

从原理上看，前者在编译阶段插入路径跟踪代码，需要知道源码且源码可编译，后者则通过运行时动态模拟指令行为，无需源码；前者对性能友好，后者对闭源程序更友好。

从实验结果来看，编译插桩在路径覆盖数量、速度以及崩溃检测效率方面表现更佳，而 QEMU 模式尽管慢但通用性强。

因此，在能获取源码的情况下，推荐优先使用编译插桩方式；而在分析黑盒程序时，QEMU 模式则是不可替代的手段。